Advanced Praktika SoSe 26 Notes

Igor Dimitrov

2026-04-21

Table of contents

Preface

This is a Quarto book.

To learn more about Quarto books visit https://quarto.org/docs/books.

Part I

Notes

1 Project Summary

1.1 Abstract

This *Fortgeschrittenenpraktikum* project investigates mixed precision iterative refinement for solving linear systems

$$Ax = b.$$

The goal is to implement a three-precision LU-based method: the LU factorization and correction solves use low precision u_f, the solution updates use working precision u, and the residuals are computed in higher precision u_r. Starting from a low-precision LU solve, the algorithm repeatedly computes $r_i = b - Ax_i$, solves for a correction d_i, and updates x_i until $|d_i|/|x_i| < u$ or a maximum iteration count is reached.

The focus is empirical accuracy, not runtime, since low precisions such as fp8, fp16, and bfloat16 are simulated via CPFloat. The experiments will study convergence, forward/backward error, condition-number dependence, precision triples (u_f, u, u_r) residual precision, and comparison with a direct working-precision solve.

Summary:

- implement three-precision vaiant of **iterative refinement for linear systems**
- study it's accuracy

1.1.1 Problem

- A linear system $Ax = b$. **iterative refinement** improves an approximate solution by repeatedly computing the residual and solving a correction equation.
  - LU factorization in low precision: u_f
  - **working arithmetic** in 'middle' precision: u
  - residual computation in high precision: u_r

> **Note**
>
> Review the definition of **accuracy** of an algorithm

- precision triples (u_r, u, u_f) with $u_r \succ u \succ u_f$:
  - i.e. u_f low precisions like: `fp8`, `fp16`, `bfloat16`, come from CPFloat:
    - `FP8`: `CPFloat<4, 4>`
    - `bfloat16`: `CPFloat<8, 8>`
    - `FP16`: `CPFloat<11, 5>`
  - u is probably the native `FP64` i.e. `double` precision

> **Note**
>
> Study the `FP8`, `FP16` and `bfloat16` formats and how they relate to template parameters in `CPfloat<>`

1.1.2 Algorithm

Algorithm: Three-precision iterative refinement

```
1. Compute the LU factorization PA = LU in precision uf.
2. Solve LU x0 = P b in precision uf; cast x0 to precision u.
3. While not converged:
   4. Compute ri = b - A xi in precision ur.
   5. Cast ri to precision u.
   6. Solve LU di = P ri in precision uf; cast di to precision u.
   7. Update xi+1 = xi + di in precision u.
```

- convergence condition: $\frac{\|d_i\|}{\|x_i\|} < u$ where u is the unit roundoff of the working precision,
- or stop after a max number of iterations.

> **Note**
>
> why is $\frac{\|d_i\|}{\|x_i\|} < u$ the convergence condition?

1.1.3 HDNUM & Own Implementation

relevant parts:

matrix and vector classes:

- `src/densematrix.hh`: `DenseMatrix<T>` with:
  - `mv`: matrix vector product
  - `mm`: matrix matrix product
  - `transpose()` etc
- `src/vector.hh`: `Vector<T>` with vector arithmetic operations (multiplication with scalar, addition) and norms
- LU decomposition in `src/lr.hh`:
  - `lr_fullpivot(A, p, q)`
  - `lr_partialpivot(A, p)`
  - triangular solves: `solveL(A, x, b)`, `solveR(A, x, b)`
  - `permute_forward` and `permute_backward`

1.1.4 Mixing Types

In a mixed precision algorithm different parts of the algorithm are instantiated / computed with different precisions, i.e. different types:

- LU factorization is done in lower precision u_f: E.g. `DenseMatrix<FP16>`
- Residual computation in higher precision u_r: E.g. `DenseMatrix<FP128>`

Therefore we must be able to convert from one type (class) like `DenseMatrix<FP16` to another type (class) like `DenseMatrix<FP128`. This conversion requires explicit construction or element-wise copying.

Managing type conversions between precisions is they key challenge. It is suggested to do this with a explicit template functions for matrices and vectors:

```
template<class T_out, class T_in>
void convert(Vector<T_out>& out, const Vector<T_in>& in)

template<class T_out, class T_in>
void convert(DenseMatrix<T_out>& out, const Vector<T_in>& in)
```

1.1.5 Methodology

- **reference solutions**: run a standard solver for $A \cdot x = b$ in `FP256` and treat the result x_{ex} as exact.
- **error metrics**:
  - report errors using norms appropriate to your algorithm

- compute **all error measures** in *a precision higher than the working precision of your algorithm*, i.e. u
- otherwise you are measuring the precision of your measurement.

> **Note**
>
> - What does **error** mean in this context? $\|x^* - x_{ex}\|_2$?
> - What is a norm that is **apropriate** to LU, simply $\|\bullet\|_2$?
> - Does Compute error measures in a precision higher than u simply mean that we compute $\|\cdot\|$ in something like `FP128`
>
> - what does "otherwise you are measuring the precision of your measurement" mean?

- experiments should be scripted so that re-running them with various precision, condition number and matrix type is convenient.

1.1.6 What Should be Studied

- **Convergence Histories**: for each precision combination (u_f, u, u_r) and several test matrices A_i with different condition numbers $k(A_i)$:
  - plot **forward error** as a function of i
  - plot **backward error** as a function of i

> **Note**
>
> review the definitions of **forward error** and **backward error**

- **Summary Across Condition Numbers**: for a wide range of $k(A)$ for each precision combinations
  - record:
    - the final achieved **forward error**
    - number of iterations
  - plot these as a function of $k(A)$: theory predicts convergence requires roughly $k(A) \cdot u_f < 1$.
- **The Role** u_r: Fix (u_f, u) and vary u_r. When does $u_r > u$ make a measurable difference

> **Note**
>
> - Normally $u_r < u$, i.e. the residual computation is done in a precision lower than the working precision.
> - What happens when we increase it, when does it make a difference?

2 Project Setup

2.1 General repository structure

The whole project is managed inside a Quarto book repository. The Quarto project contains notes, theory summaries, the final report, and the implementation code.

The planned directory structure is:

```
project-root/
   index.qmd
   report/
   notes/
   code/
       CMakeLists.txt
       external/
           hdnum/                # git submodule
               cpfloat/          # cloned inside hdnum
       include/
           mixed_ir.hpp          # algorithm
           hdnum_conversions.hpp  # convert()
           error_metrics.hpp     # forward/backward error helpers, later
       experiments/
           exp_convergence.cc
           exp_condition_sweep.cc
           exp_residual_precision.cc
       examples/
           hdnum_sandbox.cc
       tests/
           test_conversion.cc
           test_mixed_ir_sanity.cc
       scripts/
           run_all_experiments.py
           plot_results.py
       results/
           raw/
           plots/
```

Purpose of the main sub-directories:

```
external/      external dependencies, especially HDNUM
include/       own header-only algorithm and helper code
src/           optional implementation files, if needed later
examples/      small sandbox/demo programs for learning and testing
experiments/   reproducible experimental drivers for report data
tests/         correctness and sanity checks
scripts/       automation and plotting scripts
results/raw/   generated CSV or data files
results/plots/ generated figures for the report
```

2.1.1 Experimental Drivers

The will go in:

```
code/
  experiments/
      exp_convergence.cc
      exp_condition_sweep.cc
      exp_residual_precision.cc
```

They will be written in c++ because they need to instantiate HDNUM types like FP16, bfloat16, FP64, FP128, FP256, run LU, perform refinement, and compute errors.

But the overall experiment workflow will be split:

```
C++ drivers:
    run numerical experiments
    write CSV files

Python / shell scripts:
    run many driver configurations
    collect results
    make plots
```

like:

```
code/
  experiments/
      exp_convergence.cc
      exp_condition_sweep.cc
      exp_residual_precision.cc
  scripts/
      run_all_experiments.py
      plot_results.py
  results/
       raw/
       plots/
```

The cpp experiments can be written so that they can be run like:

```
./build/exp_condition_sweep --uf FP16 --u FP64 --ur FP128 --n 100 --matrix random_orthogonal
./build/exp_condition_sweep --uf bfloat16 --u FP64 --ur FP128 --n 100 --matrix random_orthog
./build/exp_condition_sweep --uf FP32 --u FP64 --ur FP128 --n 100 --matrix random_orthogonal
```

They a python or shell script can run all of those automatically.

Precision sweeps should vary precision, condition number, and matrix type, and experiments should be scripted so rerunning them is convenient. In practice, that means your experiment system should let you rerun a whole table of experiments with one command, rather than manually recompiling/editing files.

2.2 HDNUM, CPFloat and GMP

HDNUM is included as a git submodule:

```
git submodule add https://parcomp-git.iwr.uni-heidelberg.de/Teaching/hdnum.git code/external/
git submodule update --init --recursive
```

CPFloat is cloned and built inside the HDNUM top-level directory, matching HDNUM's expected layout:

```
cd code/external/hdnum
git clone https://github.com/north-numerical-computing/cpfloat.git
cd cpfloat
make lib
```

GMP is installed system-wide:

```
sudo apt install libgmp-dev
```

Because CPFloat sits inside the HDNUM submodule but is not part of HDNUM itself, untracked files are hidden locally only for that submodule:

```
cd code/external/hdnum
git config status.showUntrackedFiles no
```

and the parent repository ignores untracked content specifically inside the HDNUM submodule:

```
git config submodule.code/external/hdnum.ignore untracked
```

3 Report

3.1 This is some title